WEEK – 11

Experiment No. 11: Exception Handling

Title

Mobile Number Validation Using Exception Handling

Objective / Aim

To write a Java program to validate a mobile number using exception handling and demonstrate the use of `try`, `catch`, `throw`, `throws`, `finally`, built-in exceptions, and user-defined exceptions.

Problem Statement

Write a Java program to input a mobile number and check whether the given number is valid or invalid using exception handling.

The following conditions must be satisfied:

1. A valid mobile number must contain exactly 10 digits, and every character must be a digit from 0 to 9.
2. If the number exceeds 10 digits, raise an `ArrayIndexOutOfBoundsException` with the message `Invalid Mobile Number-ArrayIndexOutOfBounds Exception`.
3. If the number contains fewer than 10 digits, raise a user-defined `LengthNotSufficientException` with the message `Invalid Mobile Number – LengthNotSufficientException`.
4. If the number contains any character other than a digit, raise a `NumberFormatException` with the message `Invalid Mobile Number – NumberFormatException`.

Concepts Used

Concept	Description
Exception Handling	Handles runtime errors in a program.
`try`	Contains statements that may generate exceptions.

Concept	Description
`catch`	Handles exceptions thrown by the program.
`throw`	Explicitly throws an exception.
`throws`	Declares exceptions that a method may throw.
`finally`	Executes cleanup statements regardless of whether an exception occurs.
Built-in Exceptions	Predefined exceptions such as `NumberFormatException` and `ArrayIndexOutOfBoundsException`.
User-Defined Exception	A programmer-defined exception such as `LengthNotSufficientException`.

Classes Used

1. **LengthNotSufficientException** – A user-defined exception used when the mobile number contains fewer than 10 digits.
2. **MobileNumberValidation** – Contains the validation logic and the `main()` method to accept input, check the mobile number, and handle exceptions.

Algorithm / Logic

1. Start the program.
2. Create a user-defined exception named `LengthNotSufficientException`.
3. Read the mobile number as a string using the `Scanner` class.
4. Check whether the input contains any non-digit characters.
5. If any non-digit character is present, throw a `NumberFormatException`.
6. Check the length of the mobile number.
7. If the length exceeds 10 digits, throw an `ArrayIndexOutOfBoundsException`.
8. If the length is less than 10 digits, throw a `LengthNotSufficientException`.
9. If the length is exactly 10 digits and all characters are digits, display `Valid number`.
10. Use multiple `catch` blocks to handle the exceptions.
11. Use the `finally` block to close the input resource.
12. Stop the program.

Source Code

File Name: MobileNumberValidation.java

```
import java.util.Scanner;

// User-defined exception
class LengthNotSufficientException extends Exception {

    public LengthNotSufficientException(String message) {
        super(message);
    }
}

public class MobileNumberValidation {

    public static void validateMobileNumber(String mobile)
            throws LengthNotSufficientException {

        // Check whether the input contains only digits
        if (!mobile.matches("[0-9]+")) {
            throw new NumberFormatException(
                "Invalid Mobile Number - NumberFormatException"
            );
        }

        // Check whether the length exceeds 10
        if (mobile.length() > 10) {
            throw new ArrayIndexOutOfBoundsException(
                "Invalid Mobile Number-ArrayIndexOutOfBounds Exception"
            );
        }

        // Check whether the length is less than 10
        if (mobile.length() < 10) {
            throw new LengthNotSufficientException(
                "Invalid Mobile Number - LengthNotSufficientException"
            );
        }

        System.out.println("Valid number");
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter mobile number: ");
            String mobile = sc.nextLine();

            validateMobileNumber(mobile);

        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println(e.getMessage());

        } catch (LengthNotSufficientException e) {
            System.out.println(e.getMessage());

        } catch (NumberFormatException e) {
            System.out.println(e.getMessage());
```

```
        } finally {
            sc.close();
        }
    }
}
```

Explanation of the Program

1. User-Defined Exception

The `LengthNotSufficientException` class extends the `Exception` class. It is used to report an error when the mobile number contains fewer than 10 digits.

2. `try` Block

The `try` block accepts the mobile number from the user and calls the `validateMobileNumber()` method.

3. `throw` Keyword

The `throw` keyword explicitly raises an exception when the mobile number does not satisfy the validation conditions.

4. `throws` Keyword

The `validateMobileNumber()` method declares `throws LengthNotSufficientException` because the user-defined exception is a checked exception.

5. Multiple `catch` Blocks

Separate `catch` blocks handle `ArrayIndexOutOfBoundsException`, `LengthNotSufficientException`, and `NumberFormatException`.

6. `finally` Block

The `finally` block closes the `Scanner` resource after the validation process.

Compilation and Execution

Step 1: Compile the program

Open the terminal in the directory containing `MobileNumberValidation.java` and execute:

```
javac MobileNumberValidation.java
```

Step 2: Run the program

```
java MobileNumberValidation
```

Test Cases and Sample Outputs

Test Case 1: Valid Mobile Number

Input:

```
9885089465
```

Output:

```
Enter mobile number: 9885089465
Valid number
```

Explanation: The input contains exactly 10 digits, and every character is a digit.

Test Case 2: Mobile Number Exceeding 10 Digits

Input:

```
98567890121
```

Output:

```
Enter mobile number: 98567890121
Invalid Mobile Number-ArrayIndexOutOfBounds Exception
```

Explanation: The input contains 11 digits, so an `ArrayIndexOutOfBoundsException` is thrown.

Test Case 3: Mobile Number with Fewer Than 10 Digits

Input:

```
88664433
```

Output:

```
Enter mobile number: 88664433
Invalid Mobile Number – LengthNotSufficientException
```

Explanation: The input contains only 8 digits, so the user-defined `LengthNotSufficientException` is thrown.

Test Case 4: Mobile Number Containing Non-Digit Characters

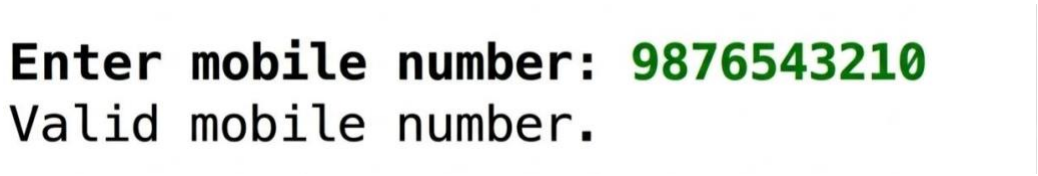
Input:

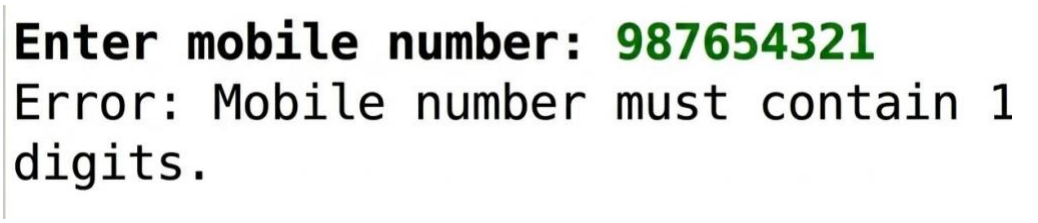
```
98ab@123
```

Output:

```
Enter mobile number: 98ab@123
Invalid Mobile Number – NumberFormatException
```

Explanation: The input contains letters and a special character. Therefore, a `NumberFormatException` is thrown.





Test Case Summary

Test Case	Input	Expected Output
1	`9885089465`	Valid number
2	`98567890121`	Invalid Mobile Number-ArrayIndexOutOfBounds Exception
3	`88664433`	Invalid Mobile Number – LengthNotSufficientException
4	`98ab@123`	Invalid Mobile Number – NumberFormatException

Result

Thus, the Java program successfully validates mobile numbers using exception handling. It identifies valid mobile numbers and handles invalid inputs using `ArrayIndexOutOfBoundsException`, `LengthNotSufficientException`, and `NumberFormatException`.

Conclusion

The program demonstrates exception handling in Java using `try`, `catch`, `throw`, `throws`, and `finally`. It also demonstrates the use of built-in exceptions and a user-defined exception to handle different mobile number validation errors.